Contents

Documentación de Casos de Uso — Aliflow 2
Vistas por rol 4
UC1 — Iniciar sesión institucional 4
UC2 — Recargar saldo 8
UC3 — Consultar menú del día 9
UC4 — Comprar almuerzo 9
UC5 — Retirar y entregar almuerzo 10
UC6 — Iniciar sesión operativa 10
UC7 — Configurar integración con sistema externo 10
UC8 — Administrar menú 11
UC9 — Consultar métricas y operación 11
UC10 — Consultar estado de sincronización 12
UC11 — Consultar detalle de venta (soporte re-emisión de comprobante) 12
UC12 — Gestionar usuarios del local 12
Cartilla de fidelidad — UC13, UC14, UC15 13
UC13 — Consultar cartilla de fidelidad 13
UC14 — Configurar programa de fidelidad del local 13
UC15 — Canjear premio de la cartilla 14
Inventario reservado y horario de retiro — UC16 y UC17 14
UC16 — Administrar inventario reservado para Aliflow 14
UC17 — Configurar horario máximo de retiro 15
Super-Admin de Aliflow — UC18, UC19 y UC20 15
UC18 — Dar de alta un local y crear su vista de proveedor 15
UC19 — Brindar soporte a los locales 16
UC20 — Administrar la plataforma 16
Casos de uso incluidos/extendidos (sub-flujos reutilizables) 16
Documentación del Diagrama de Clases — Aliflow 17
1. Paquete “Usuarios” 18
2. Paquete “Proveedor y Menú” 19
La clase `InventarioReservado` 21
3. Paquete “Wallet y Pagos” 21
4. Paquete “Órdenes” 23
5. Paquete “Fidelidad” 24
6. Paquete “Integración con ERP externo” 26
Principios SOLID aplicados 27
Patrones de diseño usados 28
Malos olores evitados 28
Documentación de Diagramas de Objetos — Aliflow 28
1. Billetera y orden de un estudiante 28
2. Integración multi-tenant con los ERP de dos locales reales (Barú/Contífico y Caramel Coffee/Alpwin) 29

Documentación del Diagrama de Componentes — Aliflow 30
Cliente 30
Aliflow API (FastAPI) 30
Capa de Integración 31
Procesamiento asíncrono 32
Sistemas externos 32
Decisiones de este diagrama que quedan abiertas 32
Documentación del Diagrama de Despliegue — Aliflow 32
Nodos 33
Flujos de comunicación relevantes 34
Decisiones que quedan abiertas en este diagrama 34

Documentación de Casos de Uso — Aliflow

Referencia: cada caso de uso indica el paso correspondiente de `../../../Flujos-Aliflow-Revision.html` (formato `{rol}-{número}`, ver nota en `../../../Hallazgos-Ingenieria-API-Generica.md` sección 5) del que se derivó. ## Actores

Actor	Tipo	Descripción
Estudiante	Primario	Usuario institucional que recarga saldo, consulta el menú, compra y retira almuerzos.
Proveedor	Primario	La **persona** que administra un local: menú, inventario, métricas, integración con su ERP y las cuentas de su propio personal. Es el gerente del local — se definió que el rol "Administrador" y el rol "Proveedor" son el mismo. Un local puede tener **varias** cuentas de Proveedor.
Operador	Primario	Valida las compras realizadas por los estudiantes y marca la entrega física del almuerzo en el punto de entrega. Un local puede tener varios. (En discusiones previas del equipo aparecía como "cajero"; el nombre oficial del rol es **Operador**.)
Proveedor de Identidad (Google OAuth)	Secundario / sistema externo	Autentica al Estudiante mediante OAuth 2.0 / OpenID Connect.
Sistema ERP del local	Secundario / sistema externo	**No es uno solo**: cada local de la universidad usa un ERP distinto (Barú → **Contífico**, Caramel Coffee → **Alpwin**, etc.). La conexión es **bidireccional** — Aliflow lee inventario/menú del ERP y le devuelve órdenes y pagos. Todos entran por la misma API genérica (patrón Adapter, ver `../../../Hallazgos-`

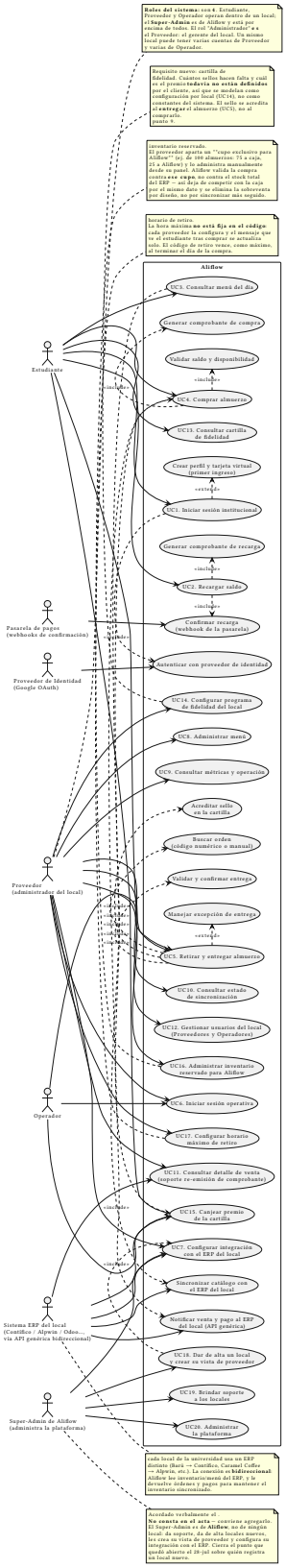


Figura 1: Vista general: todos los casos de uso del sistema

Actor	Tipo	Descripción
		`Ingenieria-API-Generica.md` sección 3).

> **Nota sobre vocabulario:** "Proveedor" se usa en dos sentidos que conviene no confundir. Como **actor/rol** es la persona que gerencia el local. Como **entidad** (clase `Proveedor` del diagrama de clases) es el local/negocio en sí — el tenant. En el diagrama de clases la persona se modela como `Administrador`, subclase de `UsuarioProveedor`; ver `../02-Modelamiento-Parte-Estatica/b-Diagrama-de-Clases.md`, sección "Usuarios".

Vistas por rol

El diagrama completo de arriba muestra el sistema entero y sirve como referencia de conjunto. A esa proporción no se lee bien impreso, así que abajo está separado en una vista por cada actor principal, con los sub-flujos que cada uno arrastra.

El Operador aparece relacionado con la compra porque el canje de un premio (UC15) incluye la creación de una orden real (UC4), que él entrega igual que cualquier otra.

UC1 — Iniciar sesión institucional

Campo	Detalle
Actor primario	Estudiante
Actor secundario	Proveedor de Identidad (Google OAuth)
Referencia	`est-1` — Autenticación
Precondición	El estudiante posee un correo institucional válido en el dominio autorizado.
Flujo principal	1. El estudiante selecciona "Iniciar sesión con Google".2. El sistema redirige al proveedor de identidad y solicita consentimiento (`<<include>>` UC1a).3. El backend valida que el dominio del correo pertenezca a la lista institucional autorizada y que el token no esté expirado.4. Se genera una sesión autenticada (token/JWT).
Flujo alternativo (extend)	**UC1b — Crear perfil y tarjeta virtual:** si es el primer ingreso del estudiante, se crea su perfil y tarjeta virtual con saldo en cero antes de continuar.
Excepción	Si el dominio del correo no pertenece a la lista institucional autorizada, se rechaza el acceso (validación crítica marcada en el flujo).
Postcondición	El estudiante queda autenticado con una sesión válida.

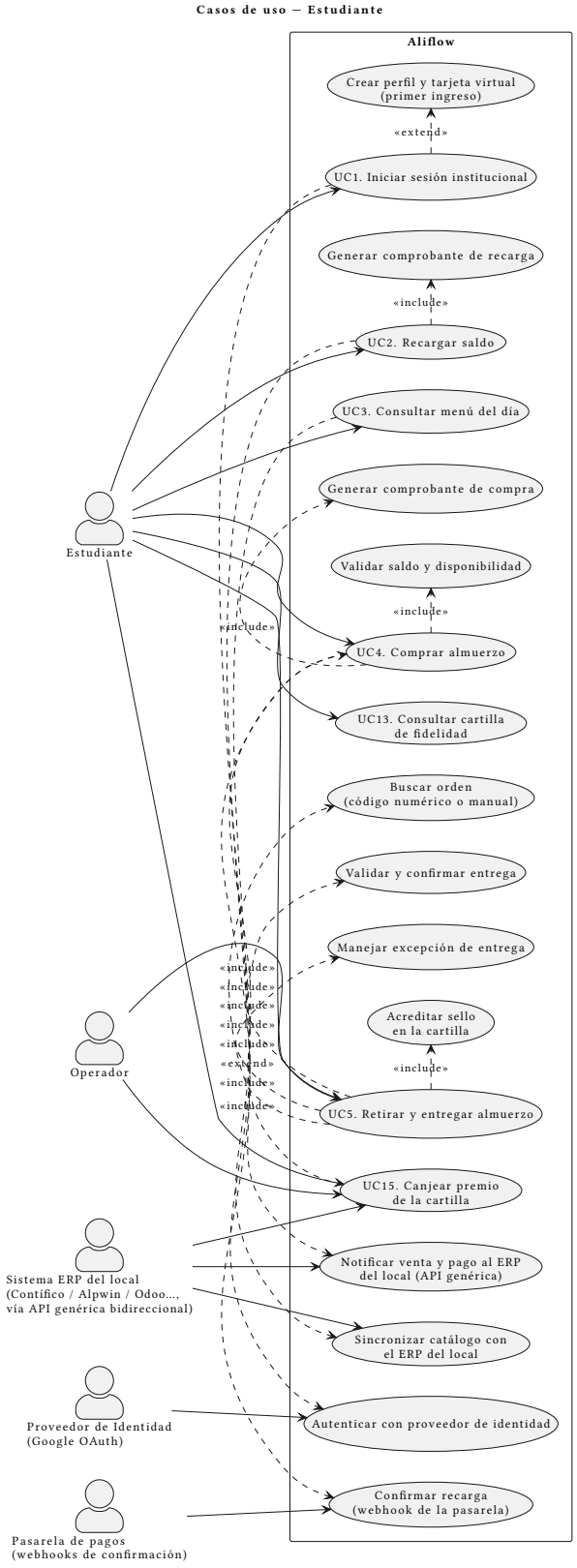


Figura 2: Casos de uso del Estudiante

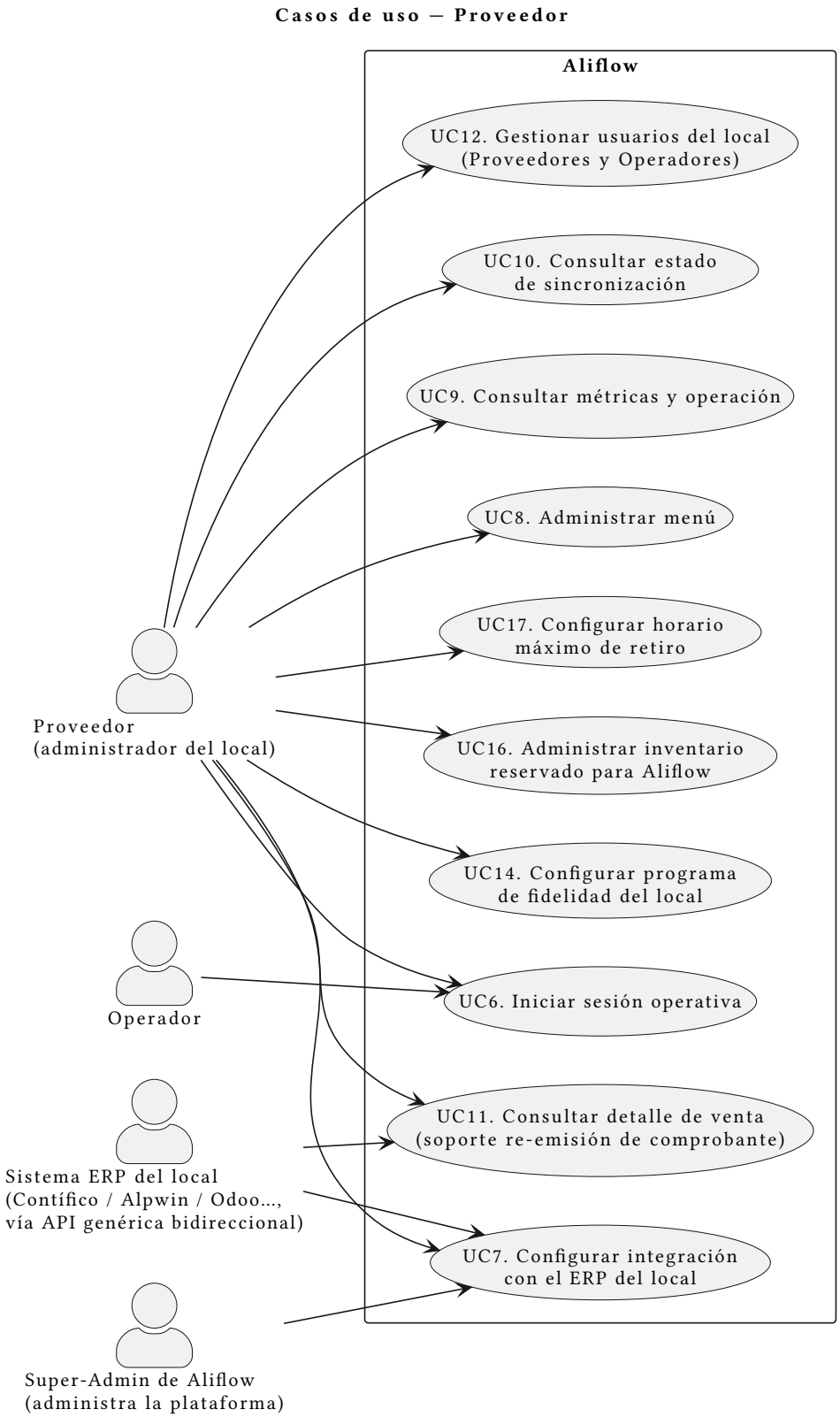


Figura 3: Casos de uso del Proveedor

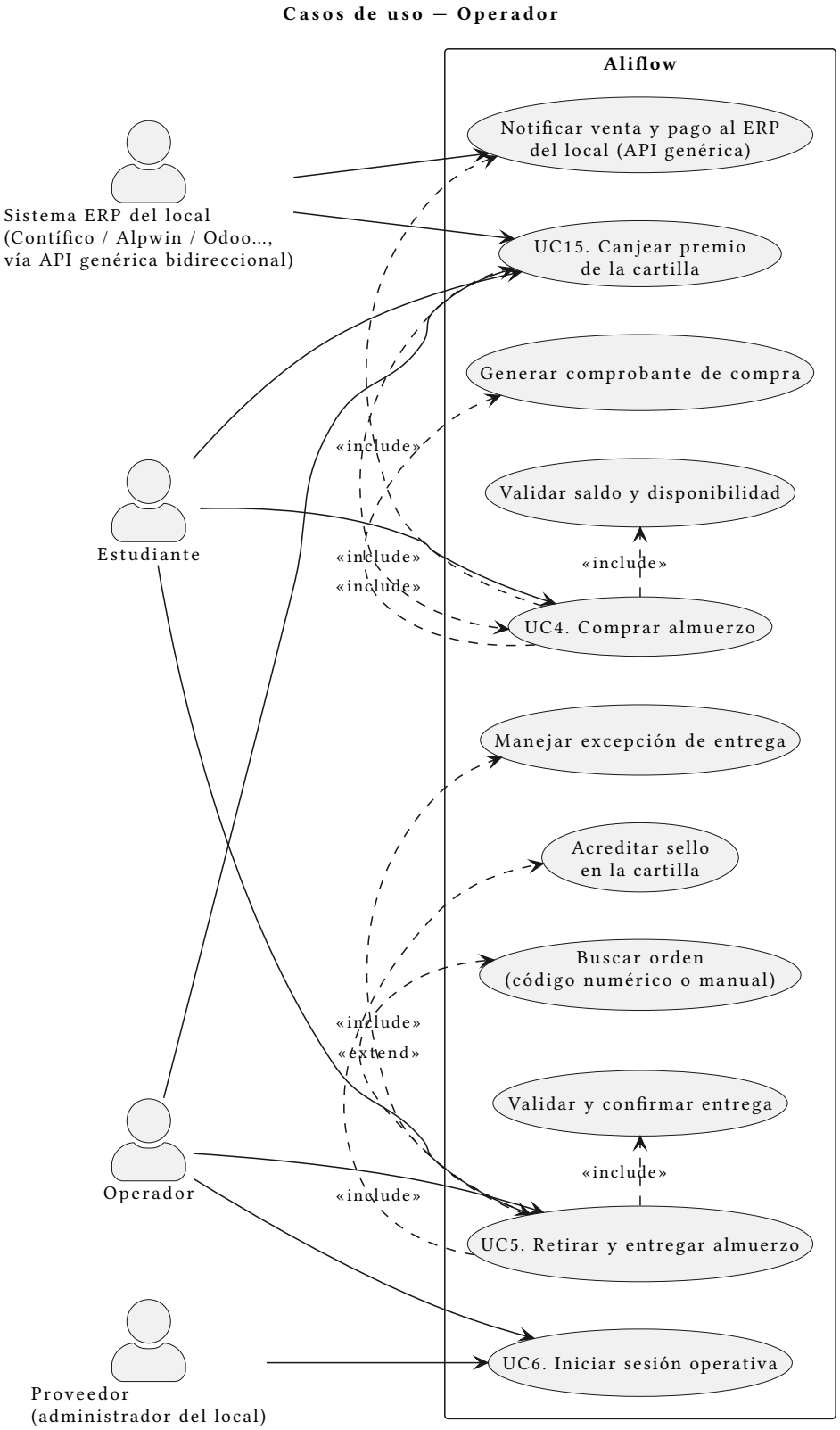


Figura 4: Casos de uso del Operador

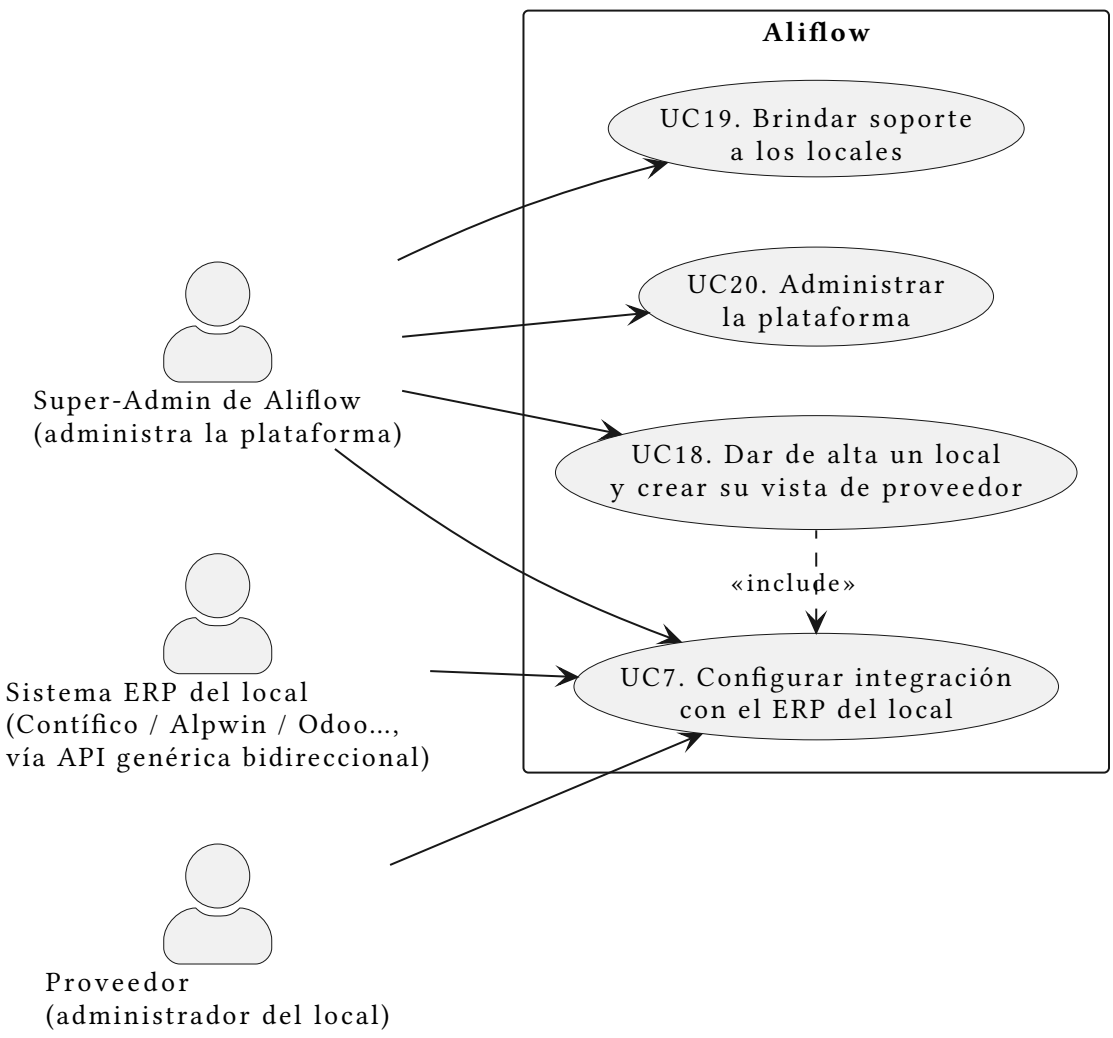


Figura 5: Casos de uso del Super-Admin de Aliflow

UC2 — Recargar saldo

Campo	Detalle
Actor primario	Estudiante
Referencia	`est-2` — Recarga de tarjeta virtual
Precondición	El estudiante está autenticado (UC1).
Flujo principal	1. El estudiante elige el **establecimiento destino** de la recarga.2. Ingresa un monto o elige uno predefinido.3. Se redirige a la pasarela de pagos.4. Al confirmarse el pago, el sistema acredita el saldo de ese establecimiento (operación atómica) e incluye la generación

Campo	Detalle
	del comprobante (<<include>> UC2a).5. El estudiante ve el saldo actualizado de ese establecimiento.
Postcondición	El saldo del estudiante en ese establecimiento queda incrementado; existe un comprobante de recarga asociado.
Regla de negocio	El saldo pertenece al establecimiento y solo puede gastarse ahí. El dinero se dirige a la cuenta de ese proveedor: Aliflow no lo recibe ni lo custodia.

UC3 — Consultar menú del día

Campo	Detalle
Actor primario	Estudiante
Actor secundario	Sistema ERP del Proveedor
Referencia	`est-3` — Consulta del menú del día
Precondición	El estudiante está autenticado.
Flujo principal	1. El estudiante accede a "Menú del día".2. El sistema sincroniza/consulta el catálogo del proveedor (<<include>> UC3a), vía la API genérica o una base local ya sincronizada.3. Se muestra el menú con disponibilidad actualizada.
Excepción	Si no hay sincronización en tiempo real, puede presentarse el caso "vendido antes de confirmar" al momento de la compra (ver UC4).
Postcondición	El estudiante visualiza platos disponibles, precio y stock.

UC4 — Comprar almuerzo

Campo	Detalle
Actor primario	Estudiante
Actor secundario	Sistema ERP del Proveedor
Referencia	`est-4` — Compra del almuerzo
Precondición	El estudiante consultó el menú (UC3) y tiene saldo disponible.
Flujo principal	1. El estudiante selecciona un plato y confirma la compra.2. El sistema valida saldo y disponibilidad (<<include>> UC4a, con revalidación de stock justo antes de confirmar).3. Si ambas validaciones pasan: se descuenta el saldo (atómico), se crea la orden en estado "Comprado", se genera el comprobante (<<include>> UC4b) y se notifica al proveedor (<<include>> UC4c).
Flujo alternativo	Si falla alguna validación (saldo o stock insuficiente), se muestra el error y no se ejecuta ningún descuento.
Excepción crítica	Concurrencia: dos estudiantes no deben poder comprar la última unidad simultáneamente — requiere bloqueo/transacción atómica (ya identificado como riesgo).

Campo	Detalle
Postcondición	Orden creada en estado “Comprado”; saldo y stock descontados; comprobante generado; ERP del proveedor notificado.

UC5 — Retirar y entregar almuerzo

Campo	Detalle
Actor primario	Estudiante y Operador (caso de uso compartido — ambos participan de la misma transacción)
Referencia	`est-6` (Estudiante) y `op-2`/`op-3`/`op-4`/`op-5` (Operador)
Precondición	Existe una orden en estado “Comprado” (UC4).
Flujo principal	1. El estudiante se presenta en el punto de entrega con su código de validación.2. El operador busca la orden (`<<include>>` UC5a — por código o búsqueda manual por nombre/ID institucional si el estudiante no tiene el código).3. El sistema valida y confirma la entrega (`<<include>>` UC5b): verifica que el estado sea “Comprado” (no “Entregado”), cambia el estado a “Entregado”, registra timestamp y operador, e invalida el código (uso único).
Flujo alternativo (extend)	**UC5c — Manejar excepción de entrega:** código inválido/ya usado (mensaje de error con hora de entrega previa), o fallo de conexión (v1 no tiene modo offline — riesgo ya documentado).
Postcondición	Orden en estado “Entregado”; código invalidado.

UC6 — Iniciar sesión operativa

Campo	Detalle
Actor primario	Proveedor, Operador
Referencia	`prov-1`, `op-1`
Precondición	El usuario tiene una cuenta con el rol correspondiente ya asignada (no es OAuth institucional, son credenciales propias del rol).
Flujo principal	1. El usuario inicia sesión con sus credenciales de rol.2. El sistema valida el rol y da acceso a la interfaz correspondiente (panel de administración para Proveedor; interfaz simplificada optimizada para uso rápido en el caso de Operador).
Postcondición	Sesión autenticada con permisos del rol correspondiente.

UC7 — Configurar integración con sistema externo

Campo	Detalle
Actor primario	Proveedor (junto con el equipo de desarrollo)
Actor secundario	Sistema ERP del Proveedor

Campo	Detalle
Referencia	`prov-2`
Precondición	El local está dado de alta en Aliflow; su ERP (Contífico, Alpwin, Odoo u otro) ya fue identificado.
Flujo principal	1. Se especifica el endpoint/mecanismo de conexión del ERP.2. Se define el mapeo de datos (productos, precios, stock) al modelo canónico de Aliflow.3. La API genérica (patrón Adapter) actúa como capa de abstracción, sin que el resto de Aliflow dependa de la implementación específica del ERP.
Postcondición	El adaptador correspondiente (`ContificoAdapter`, `AlpwinAdapter`, `OdooAdapter`, …) queda configurado y operativo para ese local, en ambos sentidos: Aliflow lee su inventario y le devuelve órdenes y pagos.
Nota de arquitectura	Este caso de uso es la materialización directa del reto técnico investigado en `../../../Hallazgos-Ingenieria-API-Generica.md` — ver secciones 3 y 4.2 (demo validado con Odoo Community).

UC8 — Administrar menú

Campo	Detalle
Actor primario	Proveedor
Referencia	`prov-3`
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor define el menú del día (plato, descripción, precio, stock inicial), manualmente o sincronizado desde su ERP.2. El sistema valida datos mínimos (precio > 0, nombre no vacío, stock ≥ 0) antes de publicar.
Postcondición	Menú publicado y disponible para consulta (UC3).

UC9 — Consultar métricas y operación

Campo	Detalle
Actor primario	Proveedor
Referencia	`prov-5`
Precondición	El proveedor está autenticado.
Flujo principal	1. El proveedor accede a su panel de métricas.2. El sistema calcula y muestra: almuerzos vendidos por día/semana, ingresos, platos más vendidos, y órdenes “Comprado” pendientes de “Entregado”, a partir del histórico de órdenes de Aliflow.
Postcondición	El proveedor visualiza el estado operativo de su negocio.

UC10 — Consultar estado de sincronización

Campo	Detalle
Actor primario	Proveedor
Referencia	`prov-4`
Precondición	El proveedor tiene su integración configurada (UC7).
Flujo principal	1. El proveedor consulta el estado de sincronización de inventario en su panel.2. El sistema muestra la última comunicación exitosa con su ERP y eventos pendientes/fallidos (tabla de reconciliación, ver arquitectura outbox en `../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4).
Postcondición	El proveedor conoce si hay ventas pendientes de reflejarse en su ERP externo.

UC11 — Consultar detalle de venta (soporte re-emisión de comprobante)

Campo	Detalle
Actor primario	Proveedor
Actor secundario	Sistema ERP del Proveedor
Referencia	`prov-6`
Precondición	Existen ventas registradas en Aliflow (UC4).
Flujo principal	1. El proveedor consulta o descarga el detalle de una venta (monto, plato, estudiante, fecha/hora) desde Aliflow.2. El proveedor re-emite la factura válida en su propio sistema contable/ERP, usando ese detalle como fuente de datos.
Postcondición	El proveedor cuenta con la información necesaria para emitir su factura fiscal real; Aliflow nunca emite un comprobante con validez tributaria.
Nota importante	Este es el caso de uso donde se detectó la contradicción entre el Acta y el flujo documentado respecto al comprobante tributario — ver `../../../Hallazgos-Ingenieria-API-Generica.md` sección 5.1. El modelo correcto es el aquí descrito: re-emisión por el proveedor, no emisión fiscal directa de Aliflow.

UC12 — Gestionar usuarios del local

Campo	Detalle
Actor primario	Proveedor (administrador del local)
Referencia	`prov-1` — "el proveedor **o personal autorizado**". Este caso de uso cierra ese vacío.
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor crea una cuenta nueva

Campo	Detalle
	para su local, eligiendo el rol: **Proveedor** (otro administrador) u **Operador**.2. Si es Operador, lo asocia a un punto de entrega.3. Puede revocar accesos o restablecer credenciales de las cuentas de su propio local.
Postcondición	El personal del local queda habilitado con el rol que le corresponde, con acceso limitado a ese local.
Regla de alcance	Un Proveedor solo puede gestionar cuentas **de su propio local**. No hay ningún rol con visibilidad sobre todos los locales.

Cartilla de fidelidad — UC13, UC14, UC15

UC13 — Consultar cartilla de fidelidad

Campo	Detalle
Actor primario	Estudiante
Referencia	Ninguna aún — requisito nuevo, no está en el flujo ni en el acta.
Precondición	El estudiante está autenticado (UC1).
Flujo principal	1. El estudiante abre su sección de fidelidad.2. El sistema muestra **una cartilla por local** en el que haya comprado: sellos acumulados sobre sellos requeridos, premio ofrecido, y fecha de expiración si aplica.3. Si alguna cartilla está `COMPLETA`, se destaca que tiene un premio disponible para canjear (UC15).
Postcondición	El estudiante conoce su avance en cada local.

UC14 — Configurar programa de fidelidad del local

Campo	Detalle
Actor primario	Proveedor (administrador del local)
Referencia	Ninguna aún — requisito nuevo.
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor activa o desactiva el programa de fidelidad de **su** local.2. Define cuántos sellos requiere la cartilla, en qué consiste el premio, cuántos sellos como máximo se pueden acumular por día, y si la cartilla caduca.3. El sistema valida los datos mínimos (sellos requeridos > 0).
Postcondición	El programa queda activo; a partir de ahí, cada entrega acredita sellos (UC5d).
Regla de alcance	El programa es **por local**, no de la plataforma: el costo del premio lo absorbe el local, así que cada uno define el suyo. Un local puede no tener programa.

UC15 — Canjear premio de la cartilla

Campo	Detalle
Actor primario	Estudiante y Operador (transacción compartida, igual que UC5)
Referencia	Ninguna aún — requisito nuevo.
Precondición	El estudiante tiene una cartilla en estado `COMPLETA` en ese local.
Flujo principal	1. El estudiante elige el plato del premio y confirma el canje.2. El sistema crea una **orden real** (`<<include>>` UC4) con `esCanje = true`, que conserva el precio del plato y le aplica un descuento del 100% rotulado como premio: se descuenta el cupo y se genera código de retiro como en cualquier compra, pero **no** se descuenta saldo.3. La cartilla pasa a `CANJEADA` mediante una actualización atómica y condicional (`WHERE estado = 'COMPLETA'`), igual que la redención del código de retiro.4. El Operador entrega el premio y confirma, como en UC5.
Flujo alternativo	Si la cartilla ya fue canjeada entre el paso 1 y el 3 (doble canje concurrente), la actualización afecta 0 filas y se informa error sin crear la orden.
Postcondición	Cartilla en `CANJEADA`; orden de canje entregada; inventario del local descontado.
Nota de integración	El canje se notifica al ERP del establecimiento como venta con descuento del 100%, no como venta de importe cero.

Inventario reservado y horario de retiro — UC16 y UC17

UC16 — Administrar inventario reservado para Aliflow

Campo	Detalle
Actor primario	Proveedor
Referencia	Acta, sección 1.3.
Precondición	El proveedor está autenticado (UC6) y tiene platos publicados (UC8).
Flujo principal	1. El proveedor consulta el cupo actualmente asignado a Aliflow por plato.2. Asigna, aumenta o reduce las unidades destinadas exclusivamente a las ventas por Aliflow.3. El sistema valida que el cupo no sea negativo ni menor al ya consumido.4. El cupo queda disponible para la compra de estudiantes (UC4).
Postcondición	Aliflow puede vender hasta el cupo asignado, con independencia de lo que ocurra en la caja del local.
Regla de negocio	**Aliflow valida la compra contra este cupo, no contra el stock total del ERP.** Si el local tiene 100 almuerzos y

Campo	Detalle
	asigna 25 a Aliflow, la aplicación deja de vender al llegar a 25 aunque el ERP reporte unidades libres.
Fundamento	Elimina la sobreventa **por diseño** en lugar de mitigarla sincronizando más seguido. Aliflow deja de competir con la caja por el mismo contador. Ver `../../../Decisiones-Pendientes-Negocios.md`, punto 10.

UC17 — Configurar horario máximo de retiro

Campo	Detalle
Actor primario	Proveedor
Referencia	Acta, secciones 6.1 y 6.2.
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor define la hora máxima hasta la que se pueden retirar los almuerzos de su local.2. El sistema la guarda en `Proveedor.horaMaximaRetiro`.3. A partir de ahí, el mensaje de confirmación que ve el estudiante al comprar se arma con ese valor, y la expiración del código de retiro se calcula con él.
Postcondición	El horario aplica solo a ese local y se refleja automáticamente en la app del estudiante.
Regla de negocio	**El horario no puede estar fijo en el código.** Cada local define el suyo. La vigencia del código termina, como máximo, al terminar el día de la compra.

Super-Admin de Aliflow — UC18, UC19 y UC20

Es un administrador **del lado de Aliflow**, no de ningún local. Es el único rol con visibilidad sobre todos los tenants.

UC18 — Dar de alta un local y crear su vista de proveedor

Campo	Detalle
Actor primario	Super-Admin de Aliflow
Precondición	Existe un acuerdo comercial con el local.
Flujo principal	1. El Super-Admin registra el local nuevo con sus datos (nombre comercial, RUC, puntos de entrega).2. Crea su vista de proveedor y la primera cuenta de Proveedor del local.3. Configura la integración con el ERP de ese local (`<<include>>` UC7).4. El local queda habilitado para publicar menú y vender.
Postcondición	El local opera como un tenant más de la plataforma.

Campo	Detalle
Qué cierra	El punto que quedó abierto el 28-jul: *"si ningún rol del sistema da de alta un local nuevo, ese paso es manual y fuera de alcance"*. **Ya no es manual ni está fuera de alcance.**

UC19 — Brindar soporte a los locales

Campo	Detalle
Actor primario	Super-Admin de Aliflow
Flujo principal	1. El Super-Admin consulta el estado de un local: órdenes, sincronización con su ERP, cupo reservado, incidencias.2. Diagnostica y resuelve el problema, o escala al equipo técnico.
Regla de alcance	Es el único rol que puede ver información de más de un local. Cada acción queda en `RegistroAuditoria`.

UC20 — Administrar la plataforma

Campo	Detalle
Actor primario	Super-Admin de Aliflow
Flujo principal	Configuración general de Aliflow: activar o desactivar locales, parámetros globales y monitoreo del estado de las integraciones de todos los tenants.

Casos de uso incluidos/extendidos (sub-flujos reutilizables)

Estos no son procesos de negocio independientes, sino pasos reutilizados dentro de los casos de uso principales (relación `<<include>>`/`<<extend>>` en el diagrama). Se documentan de forma breve:

ID	Nombre	Incluido/extiende en	Propósito
UC1a	Autenticar con proveedor de identidad	UC1 (include)	Delegar la autenticación en Google OAuth 2.0 / OIDC.
UC1b	Crear perfil y tarjeta virtual	UC1 (extend, solo primer ingreso)	Inicializar el perfil del estudiante con saldo en cero.
UC2a	Generar comprobante de recarga	UC2 (include)	Emitir constancia sin validez tributaria de la recarga.
UC3a	Sincronizar catálogo con sistema del proveedor	UC3 (include)	Obtener platos/precio/stock actualizados vía la API genérica.
UC4a	Validar saldo y disponibilidad	UC4 (include)	Revalidar saldo suficiente y stock justo antes de confirmar la compra.

ID	Nombre	Incluido/extiende en	Propósito
UC4b	Generar comprobante de compra	UC4 (include)	Emitir constancia interna sin validez tributaria (ver UC11).
UC4c	Notificar venta al sistema del proveedor	UC4 (include)	Descontar stock en el ERP externo vía la API genérica (patrón outbox si falla, ver arquitectura).
UC5a	Buscar orden (código o manual)	UC5 (include)	Ubicar la orden por código de validación o por nombre/ID institucional.
UC5b	Validar y confirmar entrega	UC5 (include)	Verificar estado, cambiar a “Entregado”, invalidar código, registrar timestamp/operador.
UC5c	Manejar excepción de entrega	UC5 (extend)	Código inválido/ya usado, o fallo de conectividad (riesgo v1 sin modo offline).
UC5d	Acreditar sello en la cartilla	UC5 (include)	Sumar un sello a la cartilla vigente del estudiante en ese local, si el local tiene programa de fidelidad activo y no se acreditó ya un sello ese día.

Documento preparado por el Grupo de el equipo de desarrollo como parte del entregable 02.a (Diagramas de casos de uso y documentación completa de cada caso de uso).

Documentación del Diagrama de Clases — Aliflow

Este diagrama modela la **lógica de negocio** del sistema (no las clases de infraestructura/ORM/framework), organizada en seis paquetes.

El modelo se presenta en **una vista por paquete** en lugar de una sola lámina. Con cuarenta clases, sus atributos y sus operaciones, un único diagrama ajustado al ancho de la página deja el texto en menos de medio milímetro de alto: existe, pero no se puede leer. La vista general de abajo cumple el papel de mapa —qué paquetes hay y cómo dependen entre sí— y cada paquete se desarrolla después con todo su detalle.

Cómo leer las vistas de detalle. Cada una muestra completas las clases de su paquete y, en **gris y con el estereotipo «contexto»**, las clases de otros paquetes con las que se relaciona. Una clase gris siempre aparece completa en la vista de su propio paquete; se la dibuja aquí solo para que la asociación no quede colgando.

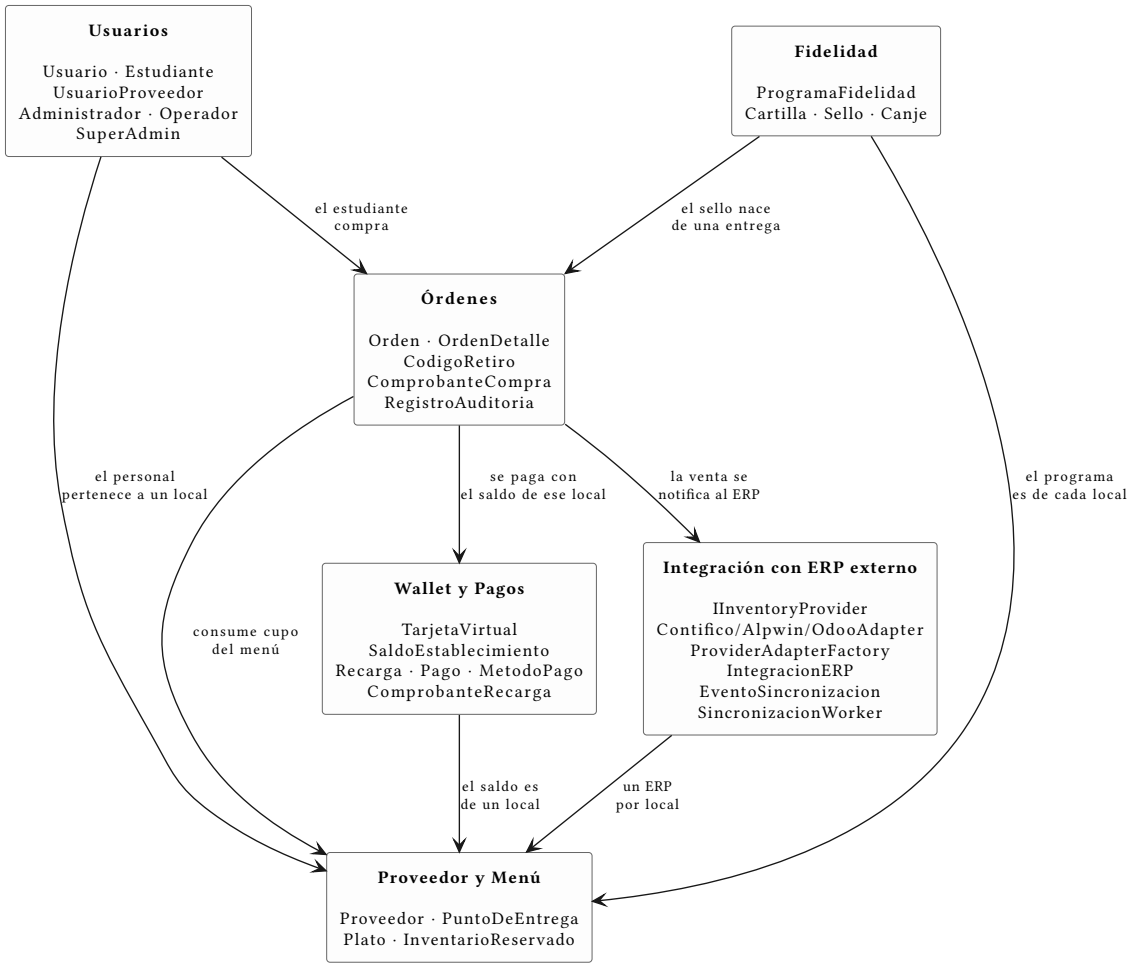


Figura 6: Vista general: los seis paquetes del modelo y sus dependencias

1. Paquete “Usuarios”

Jerarquía de herencia con **`Usuario`** como clase abstracta base (id, nombreCompleto, email, fechaRegistro, activo), especializada en:

- **`Estudiante`** — mapea al actor “Estudiante”.
- **`UsuarioProveedor`** (abstracta) — una persona con acceso al panel de un local específico (`# proveedor: Proveedor`). Se especializa en **`Administrador`**, que mapea al actor **“Proveedor”** (administra menú, métricas, la integración con el ERP del local y las cuentas del personal de su propio local, UC12), y en **`Operador`**, que valida las compras y marca la entrega física, asociado a un `PuntoDeEntrega` específico.
- **`SuperAdmin`** — extiende `Usuario` directamente, **no** `UsuarioProveedor`. La distinción es deliberada: `UsuarioProveedor` obliga a pertenecer a un local, y el Super-Admin es el único rol **sin** local, con visibilidad sobre todos los tenants. Da de alta locales nuevos, les crea su vista de proveedor, configura su integración con el ERP y brinda soporte.

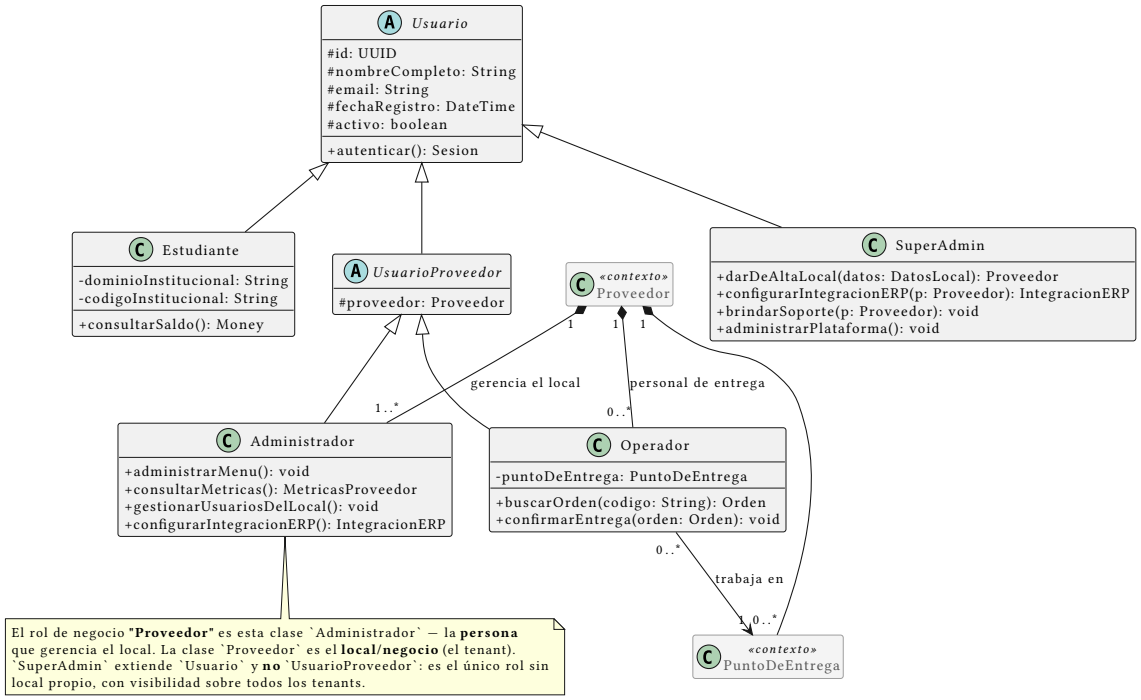


Figura 7: Paquete “Usuarios”: la jerarquía de roles y su relación con el local

Nota de vocabulario (importante para no perderse en el diagrama): la palabra “Proveedor” designa dos cosas distintas.

Tabla 23: Los dos sentidos de la palabra “Proveedor”

En el lenguaje del cliente	En el diagrama de clases	Qué es
Rol **“Proveedor”** (= “Administrador”, el gerente)	clase `Administrador`	una **persona** con cuenta en Aliflow
El **local** / proveedor de alimentación (Barú, Caramel Coffee)	clase `Proveedor`	el **negocio**: el tenant, con su menú, su ERP y su personal

Se conservó `Proveedor` como nombre de la entidad-negocio porque así se usa en el acta (“proveedores de alimentación”) y en todo el resto del modelo (`SaldoEstablecimiento`, `IntegracionERP`, `tenantId`). Renombrarla habría propagado cambios a una decena de archivos sin ganar claridad real; la tabla de arriba y una nota dentro del propio diagrama resuelven la ambigüedad.

Un local puede tener **varias** cuentas de Proveedor y varias de Operador, de ahí las cardinalidades `Proveedor "1" *-- "1..*" Administrador` (al menos un gerente) y `Proveedor "1" *-- "0..*" Operador`.

2. Paquete “Proveedor y Menú”

`Proveedor` (el tenant/negocio — en la práctica, cada local de comida de la universidad) agrega su personal, sus puntos de entrega físicos y su menú (`Plato`). `Plato.hayStock` encapsula la

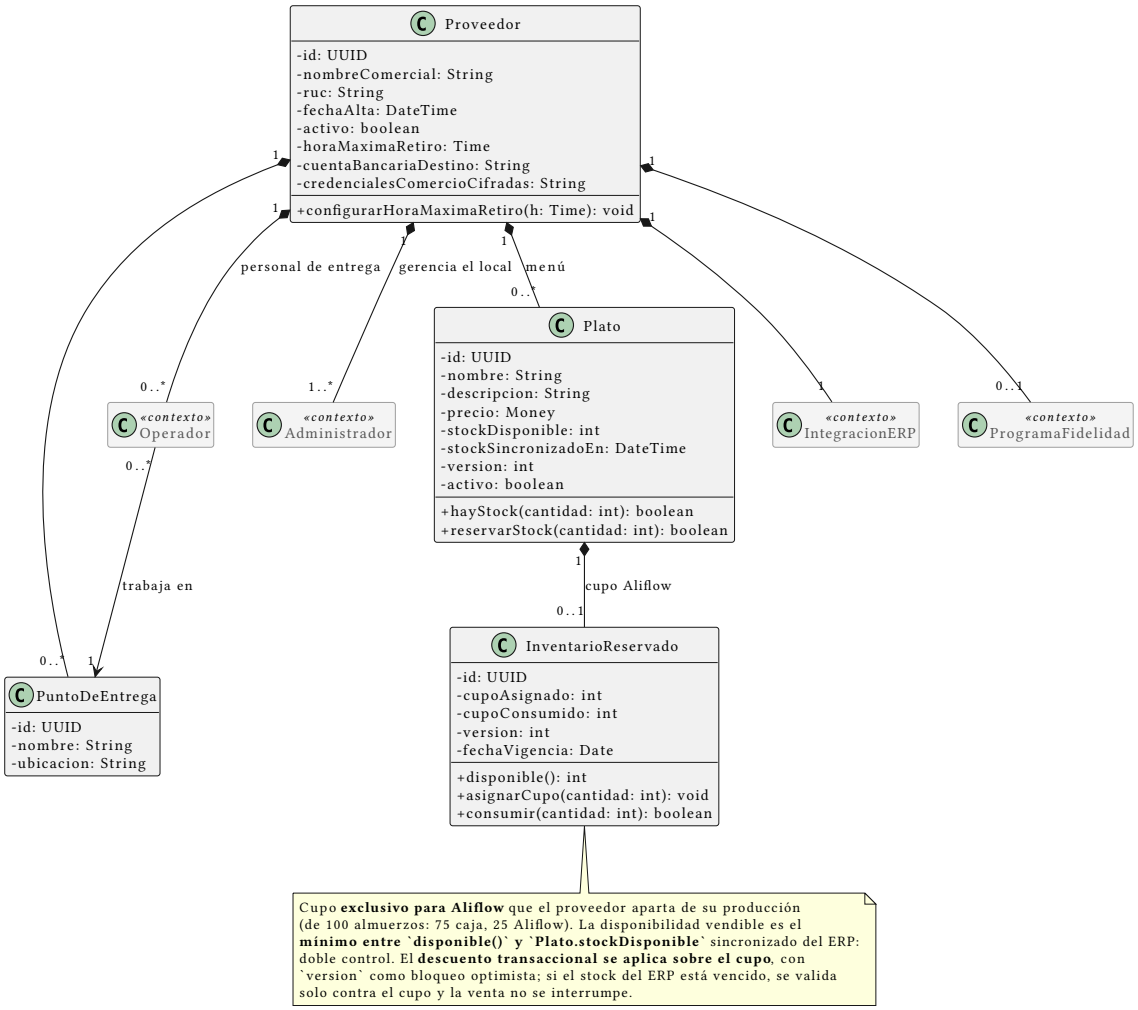


Figura 8: Paquete “Proveedor y Menú”: el local, su menú y el cupo reservado

validación de disponibilidad que en el flujo se menciona como “revalidación de stock justo antes de confirmar” (`est-4`).

Control de concurrencia. `Plato` tiene un campo `version` y un método `reservarStock(cantidad)` que implementan bloqueo optimista para el riesgo identificado desde el flujo original: dos estudiantes no deben poder comprar la última unidad simultáneamente. El detalle exacto de la transacción se completa en el diagrama de secuencia.

Validado empíricamente (ver `../../../demo-odoo/README.md` sección 7): se probó implementar este mismo bloqueo directamente contra el ERP externo (Odoo, vía RPC) y falló bajo concurrencia real — cinco hilos comprando con tres unidades de stock vendieron cinco. Esto confirma que `Plato.version`/`reservarStock` deben vivir en la base de datos propia de Aliflow y no delegarse al ERP; la misma prueba contra un dominio local con lock real (`../../../demo-odoo/plato_local.py`) sí se comportó correctamente.

La clase `InventarioReservado`

Es la clase que cambia cómo se decide si Aliflow puede vender.

El problema que resuelve. El ERP del local descuenta al instante cuando alguien compra en caja, pero Aliflow puede tardar en enterarse. En esa ventana, un estudiante compra algo que ya no existe. Son **dos escritores sobre el mismo contador** sin transacción común: sincronizar más seguido achica la ventana, no la cierra.

Cómo lo resuelve. El proveedor aparta un cupo exclusivo para Aliflow (de 100 almuerzos: 75 a caja, 25 a Aliflow) y lo administra desde su panel (UC16). **Aliflow valida la compra contra el mínimo entre `InventarioReservado.disponible()` y `Plato.stockDisponible`**, y aplica el descuento transaccional sobre el cupo. Son dos controles complementarios: el cupo acota lo que Aliflow puede vender pase lo que pase, y el stock sincronizado impide vender algo que el local ya agotó por caja.

Por qué el cupo es la pieza que sostiene el diseño: convierte un problema de consistencia distribuida —difícil, y sin solución completa cuando no se controlan los dos sistemas— en uno de **partición de recursos**, que es trivial. Aliflow es dueño exclusivo de su cupo, y por eso puede descontarlo de forma transaccional y seguir vendiendo aunque el ERP no responda. El stock sincronizado se suma como segunda cota, pero no podría ocupar ese papel: leerlo con retraso no permite descontarlo con garantías.

`Plato.stockDisponible` es el stock sincronizado desde el ERP y actúa como **cota superior**: si está vencido más allá del umbral acordado, la validación cae al cupo y la venta no se interrumpe — un ERP caído no debe tumbar la venta del local. `InventarioReservado` conserva su propio `version` para el bloqueo optimista, porque el descuento ocurre ahí y dos estudiantes sí pueden pelear por la última unidad **del cupo**.

Costo que introduce, registrado como riesgo R-20: el cupo depende de que el proveedor lo mantenga al día. Si no lo repone, Aliflow muestra "agotado" mientras el local tiene comida — y el fallo es silencioso, porque nadie reclama por lo que no puede comprar.

3. Paquete "Wallet y Pagos"

El saldo pertenece al establecimiento, no al estudiante. El estudiante recarga *para un local* y solo puede gastar ahí. El dinero va de la pasarela **directo a la cuenta de ese proveedor**; Aliflow no lo recibe ni lo custodia en ningún momento.

`TarjetaVirtual` es el contenedor que agrupa un `SaldoEstablecimiento` por cada local en el que el estudiante haya recargado. `SaldoEstablecimiento` es el saldo real en ese local: una compra en un establecimiento nunca puede consumir el saldo de otro, y no existe transferencia entre ellos.

`Recarga` lleva el establecimiento destino como dato obligatorio, y `Proveedor` guarda su cuenta bancaria y sus credenciales de comercio, porque cada local recibe directamente el dinero de sus recargas. Cada recarga registra además valor, fecha y hora, número de operación, estado de la transacción e identificador de la pasarela, y queda en histórico para auditoría y conciliación.

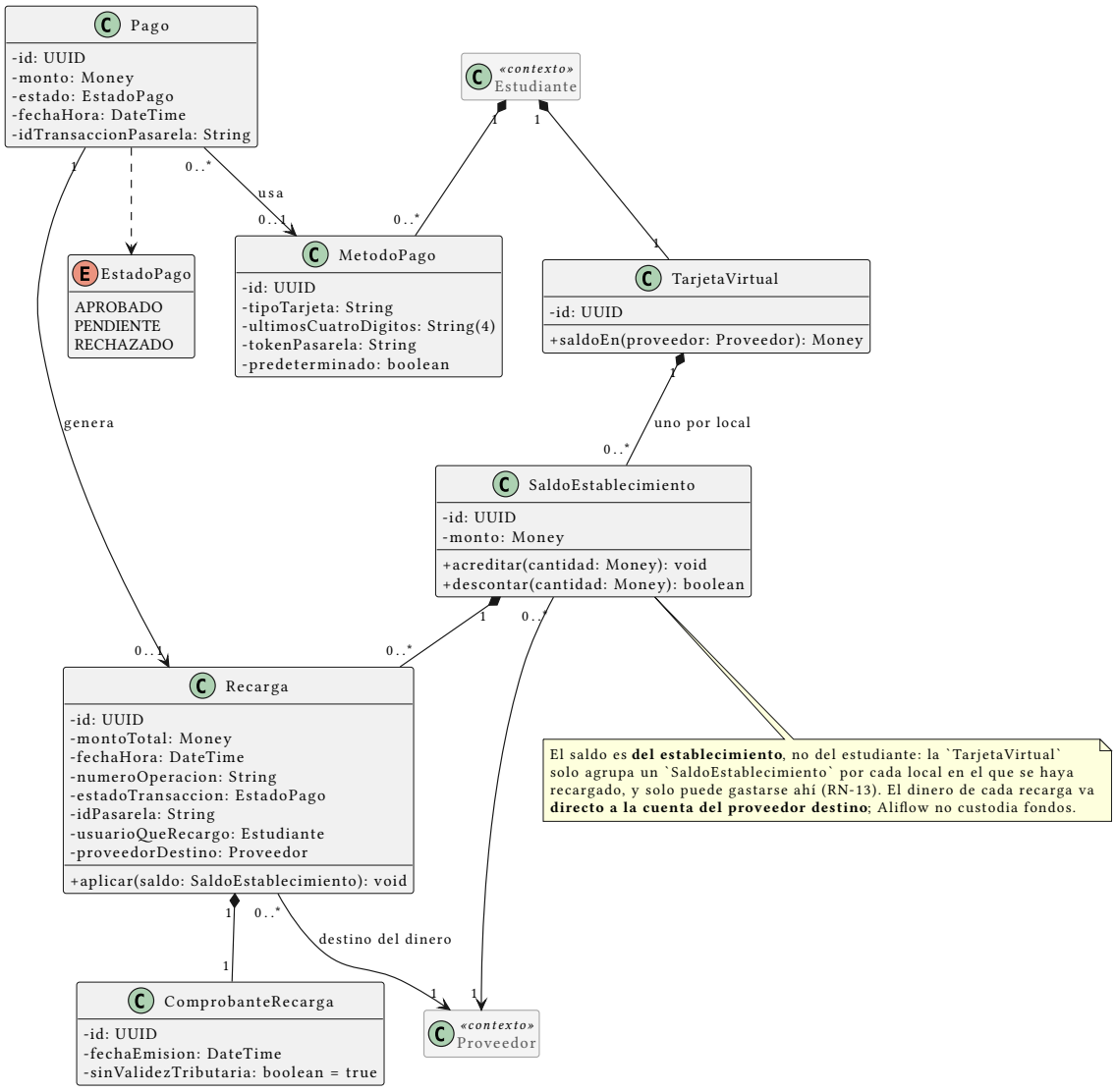


Figura 9: Paquete “Wallet y Pagos”: saldo por establecimiento, recargas y medios de pago

`Pago` (con `EstadoPago`: APROBADO/PENDIENTE/RECHAZADO) genera una `Recarga` solo si es aprobado. `ComprobanteRecarga` es el comprobante interno **sin validez tributaria**: recargar todavía no es comprar un alimento, y la factura la emite el ERP del local recién en la compra. `MetodoPago` guarda únicamente tipo de tarjeta, últimos cuatro dígitos y el token de la pasarela: Aliflow nunca almacena el número completo ni el código de seguridad.

Se eliminó el patrón Strategy `EstrategiaDistribucionRecarga`. Existía para encapsular *cómo* Aliflow repartía internamente una recarga única entre los locales. Con la recarga por establecimiento ya no hay reparto que hacer: el dinero llega directo a un único destinatario conocido desde el principio. Se retira en vez de conservarse porque un patrón sin un problema que resolver es sobre-ingeniería.

4. Paquete “Órdenes”

`Orden` agrega uno o más `OrdenDetalle` (plato + cantidad + precio unitario) y tiene un `CodigoRetiro` propio. `ComprobanteCompra` es el comprobante interno sin validez tributaria (`est-4`/`prov-6`), distinto de la factura real que el proveedor emite en su propio ERP.

`EstadoOrden` incluye **`EXPIRADO`** además de `COMPRADO`/`ENTREGADO`, para cubrir el vacío de “orden nunca retirada”: la orden expira al terminar el día de la compra. Lo único que sigue abierto es la política de reembolso del dinero de una orden expirada, y el hecho de que Aliflow nunca custodia fondos acota mucho las salidas posibles.

Formato del código. Es un **código numérico corto de seis dígitos**, no un UUID firmado. La razón es operativa: el estudiante se lo dice de viva voz al Operador, que lo digita para marcar el retiro — un UUID de 36 caracteres es impracticable para eso. Trae dos consecuencias de diseño:

1. **La unicidad se acota.** Seis dígitos son ~10^6 combinaciones: suficientes solo si la unicidad se exige entre los códigos **vigentes de un mismo local**, no globalmente ni de forma histórica.

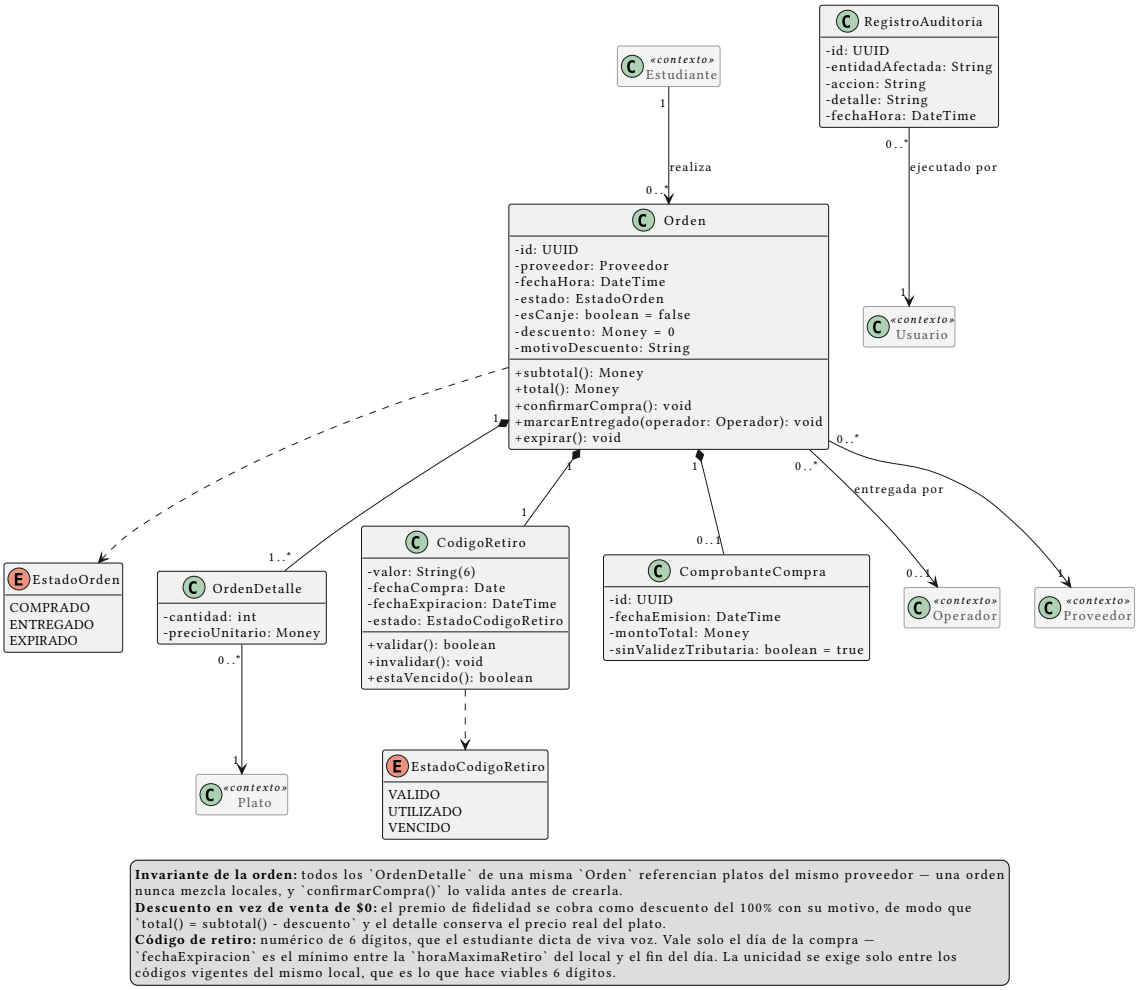


Figura 10: Paquete “Órdenes”: la compra, su detalle, su código de retiro y la auditoría

La generación reintenta ante colisión. Que el código valga un solo día es lo que mantiene ese universo pequeño.
2. **El código pasa a ser adivinable.** Un UUID firmado no se puede adivinar; seis dígitos sí. Registrado como riesgo **R-15** en `../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`, con su mitigación: límite de intentos, y el hecho de que el Operador ve físicamente al estudiante.

Regla de un solo local por orden. `Orden` tiene una asociación directa a `Proveedor` (no solo indirecta vía `OrdenDetalle → Plato → Proveedor`), con un invariante explícito en el diagrama: todos los `OrdenDetalle` de una misma `Orden` deben pertenecer a platos del mismo proveedor. Sin esa asociación el modelo permitía —sin querer— una orden con platos de proveedores distintos, lo cual habría roto el resto de la arquitectura: saldo por proveedor, un solo `tenantId` por llamada a `notifySale`, un evento de sincronización por orden. `confirmarCompra()` valida el invariante antes de crear la orden.

Auditoría. `RegistroAuditoria` responde al riesgo R-09, que pide registro de auditoría para compras y redenciones. Registra quién ejecutó `confirmarCompra()`, `marcarEntregado()` e `invalidar()` — las operaciones que cambian dinero o estado de una orden.

5. Paquete "Fidelidad"

La **cartilla de fidelidad** es una tarjeta de sellos: el estudiante acumula un sello por compra retirada y al completarla gana un premio. Siguen sin definirse solo dos valores —cuántos sellos y cuál es el premio—, y por diseño son configuración de `ProgramaFidelidad`, no constantes: cuando se definan, es cargar un dato en base de datos, no rediseñar. Lo mismo con `vigenciaCartillaDias` (si se decide que la cartilla no caduca, el campo queda nulo) y con `maxSellosPorDia`.

Cuatro decisiones de diseño sostienen el paquete:

1. **El programa es por local, no de la plataforma.** `ProgramaFidelidad` cuelga de `Proveedor`. La razón es económica, no técnica: el premio lo regala el local, así que es el local quien debe poder decidir si lo ofrece, cuántos sellos pide y qué da. Un local puede no tener programa. Como consecuencia, el estudiante tiene **una cartilla activa por local**, no una sola global.
2. **El sello se acredita al entregar, no al comprar.** `Sello` se crea dentro de `Orden.marcarEntregado()`, no de `confirmarCompra()`. Si se acreditara al comprar, un estudiante podría llenar la cartilla comprando almuerzos y nunca yendo a buscarlos — el local pagaría el premio sin haber vendido nada real. Además el sello así acompaña al acto físico, que es lo que el negocio quiere premiar.
3. **Un sello por orden, garantizado por el modelo.** La asociación `Sello --> Orden` es 1 a 1 con restricción de unicidad. Si la confirmación de entrega se reintenta (fallo de red, doble clic del Operador), la segunda inserción falla y no se acredita dos veces. Es el mismo principio de idempotencia que usa el outbox con `eventId`.
4. **Un sello por día como tope por defecto.** `maxSellosPorDia = 1`. Sin este límite, la cartilla premia volumen en vez de recurrencia, y se puede llenar en un solo día comprando varias veces el ítem más barato del menú.

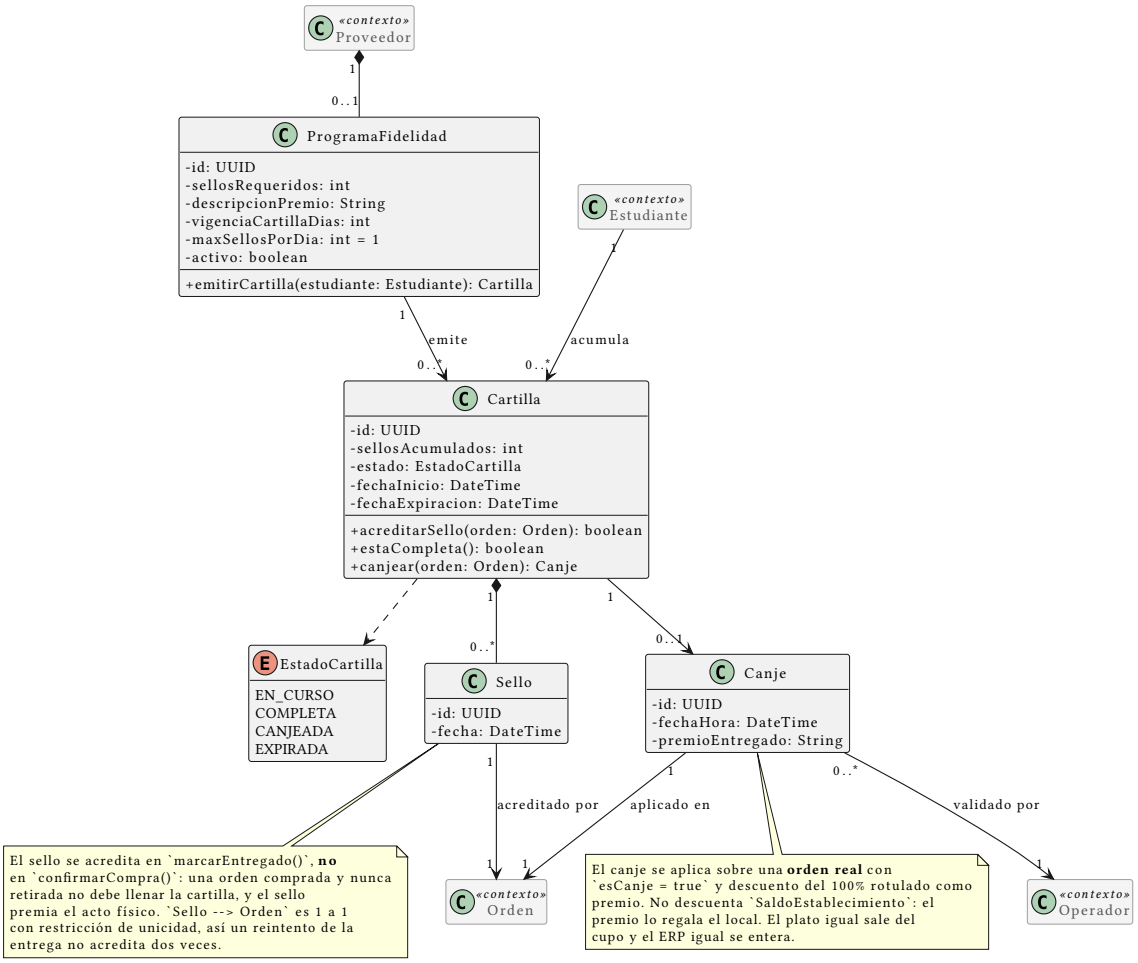


Figura 11: Paquete “Fidelidad”: programa por local, cartilla, sellos y canje

El canje toca el resto del sistema en tres lugares. Se aplica sobre una **orden real** con `esCanje = true` y **descuento del 100%** rotulado como premio, no con total $0: tiene que ser una orden de verdad porque el plato igual sale del cupo y el estudiante igual necesita un código de retiro; y conservar el precio original permite al local ver cuánto le costaron los premios, un dato que con $0 no existiría. **La wallet no se toca:** el premio lo regala el local, no se paga con `SaldoEstablecimiento`. **El ERP sí se entera,** y recibe una venta con descuento —operación normal para cualquier ERP— en vez de un importe cero que podría rechazar como error; queda por verificar contra la documentación de Contífico y de Alpwin que ambos lo admiten en línea de venta.

Concurrencia: el paso `COMPLETA → CANJEADA` usa el mismo mecanismo atómico y condicional que la redención del código de retiro (`UPDATE ... WHERE estado = 'COMPLETA'`). Si dos pestañas del estudiante intentan canjear a la vez, la segunda afecta cero filas y falla sin crear la orden.

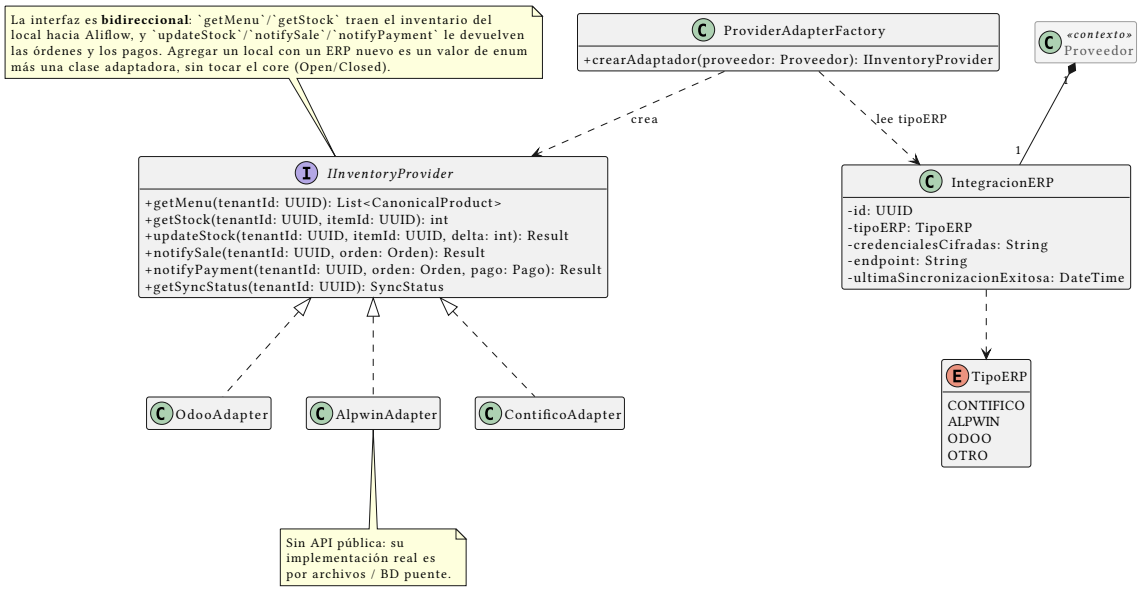


Figura 12: Paquete “Integración con ERP externo”, vista 1: el puerto y sus adaptadores

6. Paquete “Integración con ERP externo”

Materializa la arquitectura diseñada en `../../../Hallazgos-Ingenieria-API-Generica.md` (sección 3). Por su tamaño se presenta en dos vistas: el puerto con sus adaptadores, y el outbox de sincronización.

- **`IInventoryProvider`** (interfaz/puerto) — el core de Aliflow solo conoce esta abstracción, nunca un ERP concreto.
- **`ContificoAdapter`**, **`AlpwinAdapter`**, **`OdooAdapter`** — adaptadores concretos (patrón **Adapter**). `AlpwinAdapter` lleva una nota explícita: al no tener API pública, su implementación real sería por archivos o base de datos puente, no llamadas síncronas.
- **`ProviderAdapterFactory`** — patrón **Factory Method**: dado un `Proveedor`, lee su `IntegracionERP.tipoERP` y devuelve la implementación concreta correspondiente. Nadie más en el sistema necesita un `switch` sobre el tipo de ERP.

Varios adaptadores conviven en producción, no uno. Con un único ERP objetivo la factory sería casi decorativa; con un ERP por local es la pieza que hace funcionar el multi-tenant real. El diseño no cambió —es exactamente lo que Adapter + Factory está pensado para soportar— pero pasó de buena práctica defensiva a condición para que el producto exista.

La interfaz es bidireccional y eso está explícito en el diagrama. `getMenu`/`getStock` traen el inventario del local hacia Aliflow; `updateStock`, `notifySale` y `notifyPayment` devuelven al ERP las órdenes y los pagos, para que su inventario y su contabilidad queden sincronizados.

`TipoERP` es `CONTIFICO | ALPWIN | ODOO | OTRO`, con Contífico primero por ser el del local piloto. El valor `OTRO` es deliberado: agregar un local con un ERP desconocido debe ser un valor de enum más una clase adaptadora, sin tocar el core.

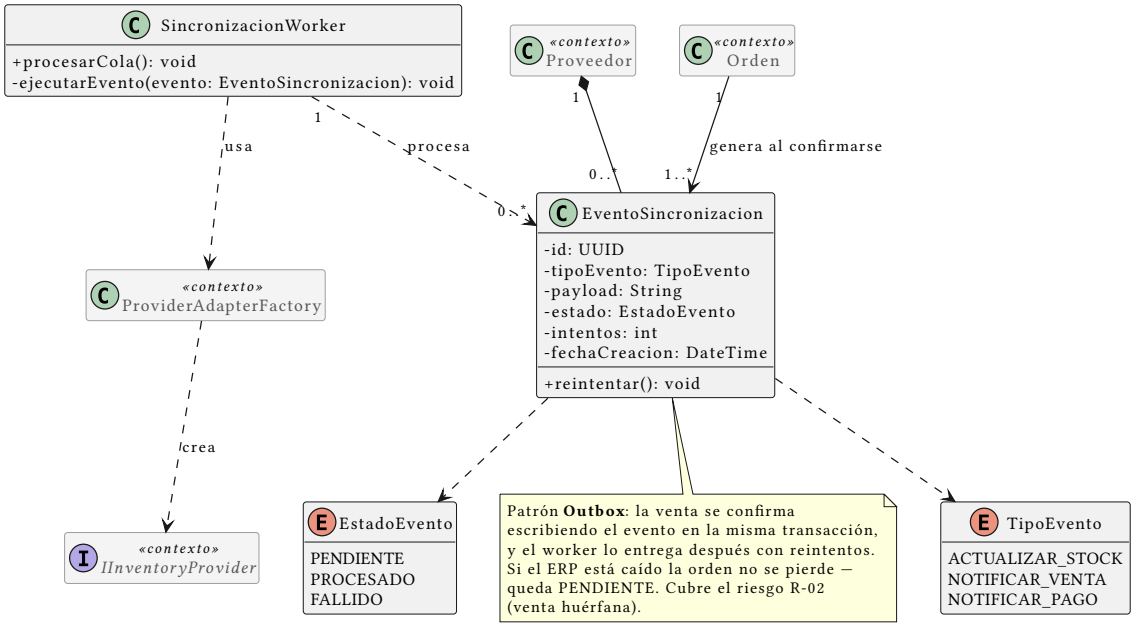


Figura 13: Paquete “Integración con ERP externo”, vista 2: el outbox de sincronización

`EventoSincronizacion` y **`SincronizacionWorker`** implementan el patrón **Outbox**: cada venta genera un evento (`TipoEvento.NOTIFICAR_VENTA`) escrito en la misma transacción que la orden, y el worker lo entrega después con reintentos (`EstadoEvento`: PENDIENTE/PROCESADO/FALLIDO). Si el ERP está caído la orden no se pierde: queda pendiente. Es la mitigación de la “venta huérfana” registrada como riesgo R-02.

Principios SOLID aplicados

Tabla 24: Principios SOLID y su punto de aplicación en el modelo

Principio	Dónde se aplica
S — Responsabilidad única	`Orden` gestiona su propio estado y ciclo de vida; la traducción a cada ERP vive exclusivamente en su adaptador; `SincronizacionWorker` solo orquesta reintentos, no lógica de negocio de la venta.
O — Abierto/cerrado	Agregar un ERP nuevo = una clase `NuevoErpAdapter` nueva, cero cambios al core (`IInventoryProvider` ya definido).
L — Sustitución de Liskov	Cualquier `IInventoryProvider` (Contífico/Alpwin/Odoo) es intercambiable sin que el código que lo usa (`SincronizacionWorker`, `Orden`) se entere de la diferencia. Con varios locales activos a la vez, esto se ejercita de verdad en producción, no solo en teoría.
I — Segregación de interfaces	`IInventoryProvider` se mantiene deliberadamente pequeña (seis métodos, todos relacionados a inventario/venta/

Principio	Dónde se aplica
	pago) — no se mezcla con responsabilidades de facturación fiscal, que quedan fuera de esta interfaz.
D — Inversión de dependencias	`ProviderAdapterFactory` y `SincronizacionWorker` dependen de la abstracción `IInventoryProvider`, nunca de `OdooAdapter`/`ContificoAdapter`/`AlpwinAdapter` directamente.

Patrones de diseño usados

- **Adapter** — resuelve directamente el reto técnico central del proyecto: integrar con ERPs heterogéneos sin acoplarse a ninguno.
- **Factory Method** (`ProviderAdapterFactory`) — evita que la lógica de "qué adaptador usar" se disperse por el código; centraliza la decisión en un solo lugar.
- **Outbox** (arquitectural, no GoF clásico) — validado técnicamente en el demo con Odoo Community (`../../../Hallazgos-Ingenieria-API-Generica.md`, sección 4.2).
- ~~**Strategy** (`EstrategiaDistribucionRecarga`)~~ — **eliminado.** Servía para encapsular cómo se repartía internamente una recarga única entre locales. Con la recarga por establecimiento ya no hay reparto, así que el patrón se quedó sin problema que resolver y se retiró en vez de conservarse por las dudas. Ver el paquete "Wallet y Pagos".

No se forzaron patrones adicionales (Singleton, Observer) donde no había un problema real que resolver: evitar ese "mal olor" de sobre-ingeniería fue una decisión deliberada. El criterio se aplica en ambos sentidos — un patrón que deja de resolver un problema se retira, porque conservarlo sería el mismo mal olor.

Malos olores evitados

- **God class**: no existe una clase "Sistema" o "AliflowService" que concentre toda la lógica — cada responsabilidad vive en la clase del dominio que le corresponde.
- **Primitive obsession**: `CodigoRetiro` y los comprobantes son objetos propios, con su propia validación, en vez de campos sueltos tipo `String codigo` regados por otras clases.
- **Acoplamiento a implementación externa**: ninguna clase de dominio (`Orden`, `Plato`, `Proveedor`) importa o conoce un SDK específico de Odoo o Contífico — todo pasa por `IInventoryProvider`.

Documentación de Diagramas de Objetos — Aliflow

Dos instantáneas (snapshots) de instancias concretas, cada una ilustrando un aspecto medular del diseño que el diagrama de clases (`../../uml/diagrama-clases.puml`) por sí solo no deja tan claro.

---

1. Billetera y orden de un estudiante

Fuente: `../../uml/objeto-billetera-orden.puml`

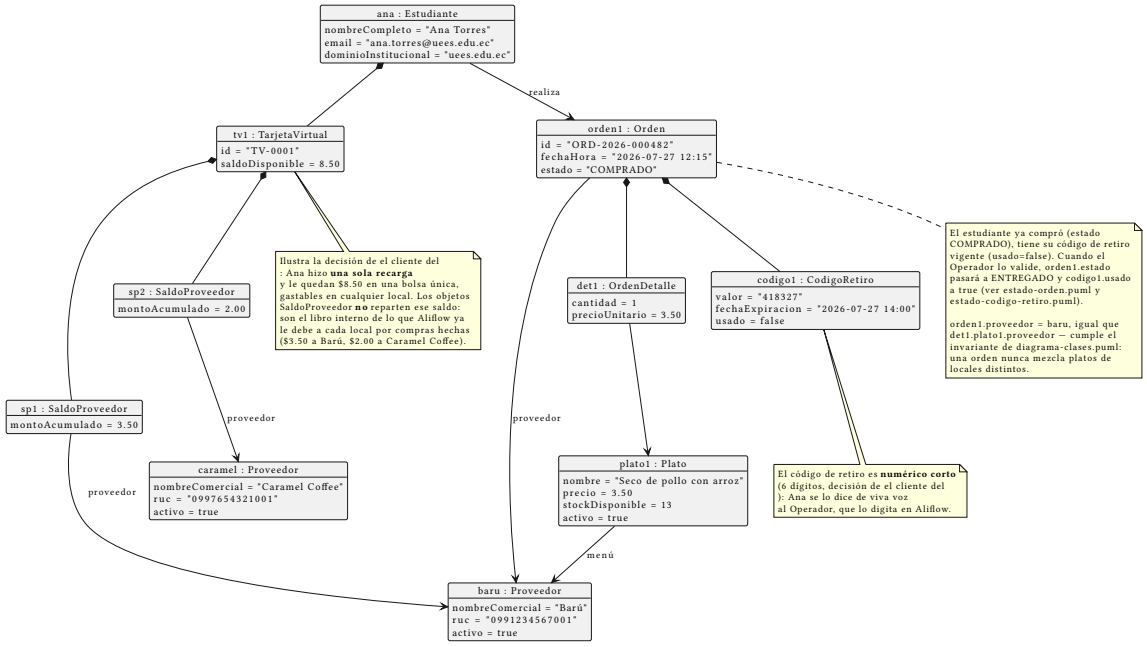


Figura 14: Diagrama de objetos: billetera y orden

Qué ilustra: cómo funciona en concreto la decisión de negocio sobre la recarga, que en el diagrama de clases solo se ve como una cardinalidad `1 -- 0..*`. Ana Torres (`Estudiante`) tiene una única `TarjetaVirtual` con **$8.50 en una bolsa común**, gastables en cualquier local — hizo una sola recarga, no una por proveedor. Los dos objetos `SaldoProveedor` colgando de esa tarjeta **no reparten** ese saldo: son el libro interno de lo que Aliflow ya le debe a cada local por compras hechas ($3.50 a Barú, $2.00 a Caramel Coffee). Esa es la lectura que el equipo de desarrollo le dio a "Aliflow distribuye internamente a todos los proveedores", y está marcada como interpretación pendiente de confirmar en `../../../Decisiones-Pendientes-Negocios.md`.

La orden (`ORD-2026-000482`) está en estado `COMPRADO`, con su `CodigoRetiro` todavía sin usar — la instantánea representa el momento justo después de la compra y antes del retiro físico del almuerzo. Ese código (`418327`) refleja el otro cambio del 28-jul: **numérico corto de 6 dígitos**, no UUID.

2. Integración multi-tenant con los ERP de dos locales reales (Barú/Contífico y Caramel Coffee/Alpwin)

Fuente: `../../uml/objeto-integracion-erp.puml`

Qué ilustra: la razón de ser de toda la capa de integración, ahora con los dos locales reales confirmados por el cliente conviviendo en el mismo snapshot:

- **Barú** tiene su `IntegracionERP` con `tipoERP = CONTIFICO`, un SaaS con API REST documentada. Su evento de venta se procesa **al primer intento** (`PROCESADO`).
- **Caramel Coffee** tiene `tipoERP = ALPWIN`, un sistema sin API pública conocida. Su evento **agota 3 reintentos y queda FALLIDO**.

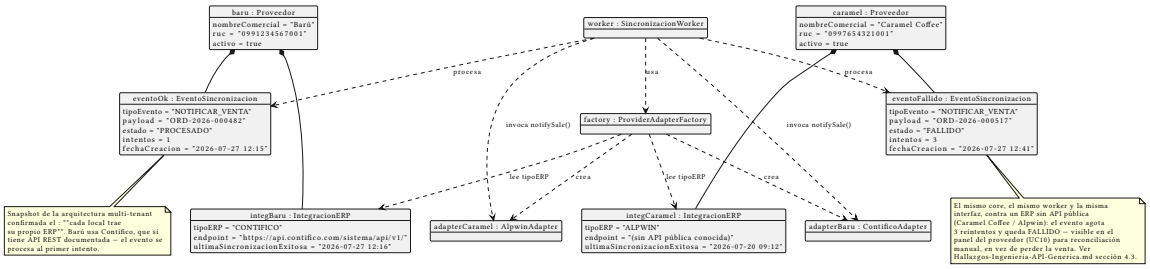


Figura 15: Diagrama de objetos: integración ERP

La misma `ProviderAdapterFactory`, el mismo `SincronizacionWorker` y el mismo contrato `IInventoryProvider` sirven a los dos casos — que es exactamente lo que el patrón Adapter debía demostrar. Antes de la corrección de el cliente, este diagrama asumía que Barú usaba Alpwin y mostraba un único caso fallido; el escenario real es mejor de lo que creíamos (el local piloto es el fácil) y a la vez más exigente (hay que sostener varios ERP a la vez, no uno).

El evento fallido no es un caso de error hipotético: es exactamente el escenario que el patrón Outbox (`../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4) fue diseñado para manejar sin perder la venta — queda visible para reconciliación manual (caso de uso UC10, `../02-Modelamiento-Parte-Estatica/a-Casos-de-Uso.md`) en vez de desaparecer silenciosamente.

Ambos diagramas usan datos de ejemplo — no corresponden a transacciones reales.

Documentación del Diagrama de Componentes — Aliflow

Este diagrama incorpora, por primera vez en la documentación formal del proyecto, el **stack tecnológico** recuperado de investigación previa del equipo (conversación revisada): Next.js en el cliente, FastAPI en el backend, PostgreSQL como base de datos, y Redis/Celery para procesamiento asíncrono. Hasta ahora esta información solo existía en una conversación externa — queda aquí formalizada como parte del diseño. ## Componentes principales

Cliente

- **Aliflow Web (Next.js + TypeScript)** — única aplicación web que sirve a los **3 roles** del sistema (Estudiante, Proveedor, Operador. No hay apps nativas separadas por rol; la diferenciación de interfaz ocurre por rol de sesión, no por despliegue distinto.

Aliflow API (FastAPI)

Organizada en módulos, cada uno mapeado a los paquetes del diagrama de clases: - **Módulo de Autenticación** — implementa UC1 (OAuth institucional) y UC6 (login operativo); es el único módulo que habla con el **Proveedor de Identidad** externo. - **Módulo de Wallet** — implementa UC2 (recarga); es el único módulo que habla con la **Pasarela de Pagos**. - **Módulo de Órdenes** — implementa UC4/UC5 (compra, retiro); publica eventos a la cola cuando se confirma una venta. - **Módulo de Proveedores y Menú** — implementa UC7/UC8 (integración, administración de menú). - **Módulo de Fidelidad** — implementa UC13/UC14/UC15 y el sub-flujo UC5d: cartillas,

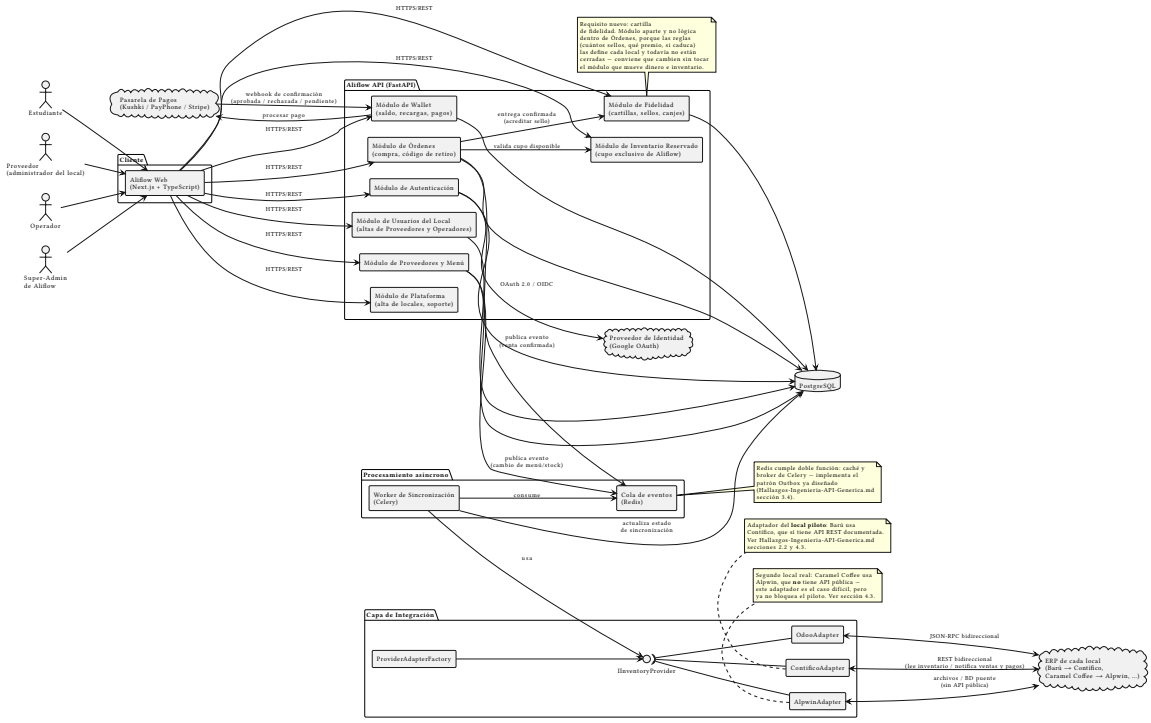


Figura 16: Diagrama de componentes de Aliflow

sellos y canjes. Se modela como módulo aparte y no como lógica dentro de Órdenes por una razón concreta: las reglas del programa (cuántos sellos, qué premio, tope diario, si caduca) las define cada local y todavía no están cerradas. Conviene que puedan cambiar sin tocar el módulo que mueve dinero e inventario. La flecha `Órdenes → Fidelidad` refleja que el sello se acredita cuando la entrega se confirma. - **Módulo de Usuarios del Local** — implementa UC12: permite que un Proveedor (administrador del local) dé de alta y revoque a otros Proveedores y a los Operadores de **su propio** local. Reemplaza al antiguo "Módulo de Administración", que existía para un rol de super-admin de plataforma ya descartado por el cliente.

Todos los módulos persisten en **PostgreSQL** directamente. No se muestra una capa de repositorio separada: a ese nivel de detalle correspondería un diagrama de clases de infraestructura, fuera del alcance de la lógica de negocio que modela este documento.

Capa de Integración

Es la traducción directa a componentes del patrón Adapter ya diseñado (`../../../Hallazgos-Ingenieria-API-Generica.md` sección 3, `../../uml/diagrama-clases.puml`): la interfaz **IInventoryProvider** (notación de lollipop) es implementada por `ContificoAdapter`, `AlpwinAdapter` y `OdooAdapter`, y `ProviderAdapterFactory` decide cuál instanciar **según el local**. Ningún otro componente del sistema depende de un adaptador concreto — solo de la interfaz.

Esta capa dejó de ser una previsión y pasó a ser un requisito duro con la confirmación de el cliente: **cada local de la universidad usa un ERP distinto** (Barú → Contífico, Caramel Coffee →

Alpwin, etc.), así que en producción van a convivir varios adaptadores simultáneamente, no uno solo. Las flechas hacia el ERP se dibujan **bidireccionales**: Aliflow lee inventario y menú desde el ERP del local, y le devuelve las órdenes y los pagos para que su inventario quede sincronizado.

Procesamiento asíncrono

- **Cola de eventos (Redis)** — recibe eventos publicados por Órdenes y por Proveedores/Menú (venta confirmada, cambio de stock).
- **Worker de Sincronización (Celery)** — consume la cola, usa la Capa de Integración para notificar al ERP externo, y actualiza el estado de sincronización en la base de datos. Es la implementación concreta del patrón Outbox.

Sistemas externos

- **Proveedor de Identidad (Google OAuth)** — ya documentado desde el flujo original (`est-1`).
- **Pasarela de Pagos (Kushki / PayPhone / Stripe)** — nueva incorporación formal; son las opciones que se manejaron en la investigación previa del equipo para el riesgo R-06 (`../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md`) sobre limitaciones del ambiente de pruebas de pagos.
- **ERP de cada local** — no es un solo sistema: Barú usa **Contífico** (API REST documentada, el caso fácil y el del local piloto) y Caramel Coffee usa **Alpwin** (sin API pública, integración por archivos/BD puente). Ver notas en el diagrama y `../../../Hallazgos-Ingenieria-API-Generica.md` sección 4.3.

Decisiones de este diagrama que quedan abiertas

1. **Elección final de pasarela de pago** (Kushki vs. PayPhone vs. Stripe) — no se ha decidido, solo se documentan como candidatas evaluadas previamente.
2. **Redis con doble rol** (caché + broker de Celery) es una simplificación de infraestructura razonable para el alcance del proyecto universitario; en un entorno de producción más grande podría separarse en dos servicios.
3. El **Módulo de Administración** expone los casos de uso UC12-UC14 que el equipo de desarrollo propuso pero el cliente no ha validado — se incluye aquí por consistencia con el diagrama de clases, no porque su alcance esté cerrado.

Documentación del Diagrama de Despliegue — Aliflow

Este diagrama traduce el diagrama de componentes (`../../uml/diagrama-componentes.puml`) a infraestructura física/de hosting concreta, combinando tres fuentes reales:

1. Las opciones de despliegue investigadas previamente por el equipo (Vercel, Render/Fly.io/Railway/AWS, Supabase/AWS RDS).
2. La infraestructura que **ya se levantó y probó** en el demo técnico con Odoo Community (`../../../Hallazgos-Ingenieria-API-Generica.md` sección 4.2).
3. La confirmación de el cliente de que **cada local trae su propio ERP**: Barú usa Contífico (SaaS, con API REST) y Caramel Coffee usa Alpwin (on-premise, sin API pública) — sección 4.3 del mismo documento.

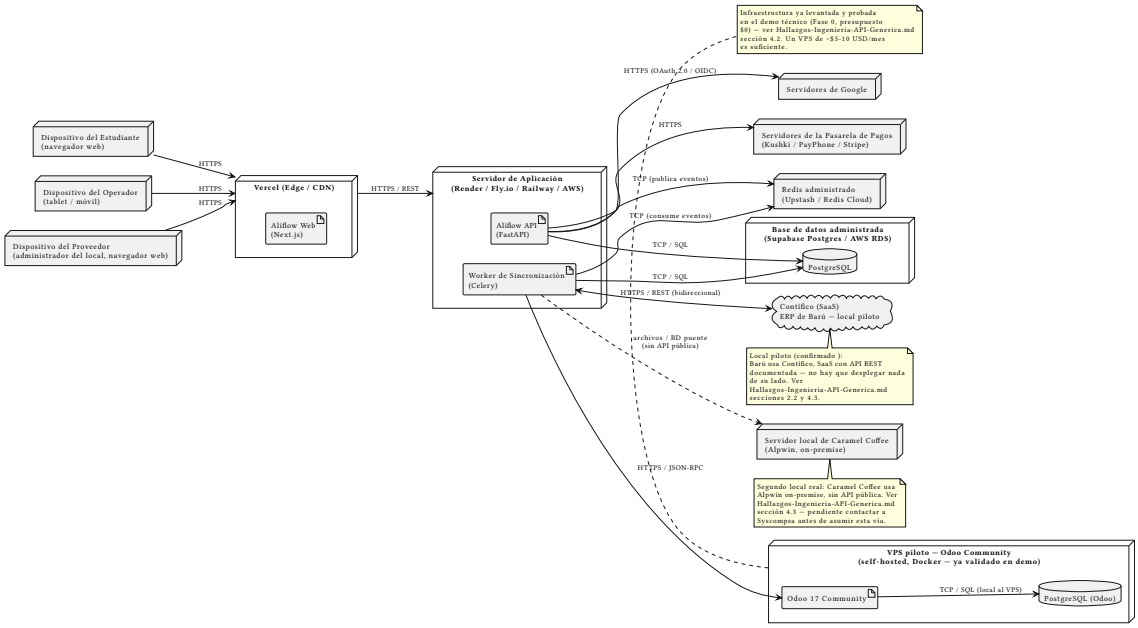


Figura 17: Diagrama de despliegue de Aliflow

Nodos

Nodo	Contenido	Notas
Dispositivos (Estudiante, Operador, Proveedor)	Navegador web / tablet	Una sola aplicación web (no apps nativas separadas); el flujo ya documentaba que el Operador usa "interfaz simplificada… móvil o tablet" (`op-1`).
Vercel (Edge/CDN)	`Aliflow Web (Next.js)`	Hosting típico para Next.js — despliegue estático/edge con baja latencia.
Servidor de Aplicación (Render/Fly.io/Railway/AWS)	`Aliflow API (FastAPI)` + `Worker de Sincronización (Celery)`	Se muestran como dos artefactos en el mismo nodo lógico, aunque en producción normalmente correrían como dos servicios/procesos escalables por separado.
Redis administrado (Upstash/Redis Cloud)	—	Cache + broker de Celery (patrón Outbox).
Base de datos administrada (Supabase Postgres/AWS RDS)	`PostgreSQL`	Persistencia del dominio de Aliflow.
VPS piloto — Odoo Community	`Odoo 17 Community` + `PostgreSQL (Odoo)`	**Esta es la única infraestructura de este diagrama que ya existe y fue probada realmente** (`docker-compose.yml` en `../../../demo-odoo/`) — no es una propuesta teórica.
Contífico (SaaS)	ERP de Barú, el local piloto	Fuera del control de Aliflow, pero no hay que desplegar nada: es un servicio en la nube con API REST documentada.

Nodo	Contenido	Notas
Servidor local de Caramel Coffee	Alpwin (on-premise)	Fuera del control de Aliflow — es la infraestructura de ese local, no la nuestra.
Servidores de Google / Pasarela de Pagos	—	Terceros, fuera de nuestro control.

Flujos de comunicación relevantes

- **Estudiante/Operador/Proveedor → Vercel → API**: todo el tráfico de usuario pasa por HTTPS/REST, sin excepción.
- **API/Worker → PostgreSQL**: persistencia síncrona vía SQL.
- **API → Redis (publica) / Worker → Redis (consume)**: implementación real del patrón Outbox — la API nunca llama directamente al ERP externo, solo encola el evento.
- **Worker → Odoo (JSON-RPC)**: el mismo protocolo ya usado y depurado en el demo (recordar el hallazgo de que hubo que usar JSON-RPC en vez de XML-RPC por el problema de serializar `None`).
- **Worker ↔ Contífico (HTTPS/REST, bidireccional)**: línea sólida — la API existe y está documentada; falta únicamente que Barú entregue las credenciales (riesgo R-01).
- **Worker → Alpwin (archivos/BD puente)**: línea punteada — a diferencia de las otras dos, esta vía **no está implementada ni confirmada todavía**; depende de lo que responda Syscompsa (pendiente en el checklist de `../../../Hallazgos-Ingenieria-API-Generica.md`).

Decisiones que quedan abiertas en este diagrama

1. **Proveedor de hosting final para la API** (Render vs. Fly.io vs. Railway vs. AWS) — no se ha decidido, solo se documentan como candidatas ya investigadas.
2. **Si el Worker corre como proceso separado o dentro del mismo contenedor que la API** — aquí se muestran como dos artefactos independientes (más correcto para escalar cada uno según su carga), pero es una decisión de implementación pendiente.
3. **La conexión real con Alpwin** — este diagrama asume la vía de archivos/BD puente como hipótesis de trabajo; podría cambiar completamente si Syscompsa ofrece algo distinto, o si el local que lo usa (Caramel Coffee) decide migrar (ver sección 4.3 del hallazgo de el equipo de desarrollo). Ya **no bloquea el piloto**, porque el local piloto es Barú con Contífico.
4. **El diagrama muestra dos ERP externos, pero el modelo admite N** — se dibujan Contífico y Alpwin porque son los dos locales confirmados; cada local nuevo agrega un nodo externo más, sin cambiar nada de la infraestructura de Aliflow.